

PyQCU 仓库分析报告（合并版）

Lattice QCD 的 Python/Cython 库：结构分析 · 核心算法剖析 · 多重网格深剖 · 全量改动概览

opencode (analy / pure / diff)

2026-08-18

摘要

本合并报告汇集对 PyQCU (/root/PyQCU) 仓库的四份分析产物，按「先结构、后算法、再深剖、末沿革」的顺序编排，便于一次性审阅：

目录

1 本合并报告包含的内容

1. **全库结构分析 (analy)** ——主体结构 / 各部分关系 / 项目思路三视角，覆盖参数协议、调用生命周期、张量布局、多线程多卡、h5py 持久化与测试体系。
2. **核心算法穷尽剖析 (pure)** ——代码—物理交叉性质判定；穷尽枚举 8 候选（深挖 7 / 浅析 1 / 排除 3），含 Wilson/Clover dslash、BiStabCG、多重网格、stout、33-tensor、CUDA/MPI、Cython 桥的双向映射。
3. **多重网格 V-cycle 15 步深剖 (pure 补充)** ——按代码工作框架 A1-A15 全流程重现，含 Galerkin 粗化、R3 fix、自动重启、5-stream 的证据直引。
4. **全量改动概览 (diff, init→stab27)** ——初始提交 0204fa5 到最新标签 stab27 的 2138 文件 / +1.16M 行改动结构化分析，含自有库演进与边界检查。

2 阅读指引

- 想了解「仓库怎么组织」→ 第 1 份 (analy)。
- 想了解「最强竞争力是什么」→ 第 2 份 (pure)。
- 想深入「多重网格如何工作」→ 第 3 份 (pure 补充, 15 步)。
- 想了解「仓库如何演进」→ 第 4 份 (diff)。

3 产物与验证

四份子报告均经 xelatex 双遍编译，交付前 Overfull 与 Float too large 计数均为 0。证据均带文件：行号，关键片段直引仓库原文件（片段见各子报告）。

4 说明

本报告各子报告保留独立标题页与目录，合并后按上述顺序连续编排；如需交叉检索，请使用 PDF 书签/目录。所有分析为只读，未改动仓库任何文件（git 提交请用户自行操作）。

PyQCU 全库结构分析

Lattice QCD 的 Python/Cython 库：CUDA 加速 Dirac 算子与多重网格求解器

opencode analy

2026-08-18

摘要

本报告对 PyQCU 仓库（/root/PyQCU）进行证据驱动的全库结构分析，覆盖两层执行层级、三张扁平张量桥接协议、调用生命周期不变量、张量布局约定、多线程多卡 MultiGrid 路径、h5py 持久化体系与测试体系。三视角结论：主体结构呈「纯 Python 层 Cython 桥 C++ CUDA 后端」三层；各部分以参数协议 `params/argv/set_ptrs` 与 `_SET_PLAN_` 计划选择织成依赖网；项目思路是用双路径（参考实现 + 生产后端）兼顾正确性与性能，并以一线程一卡并行突破单机多卡瓶颈。分析基于实测文件统计与逐文件行号证据，覆盖表闭合。

目录

1	仓库概览	2
1.1	规模统计与覆盖	2
2	主体结构	3
3	各部分关系	3
4	项目思路	4
4.1	设计意图	4
4.2	演进脉络	4
4.3	典型工作流	4
5	主题分析	4
5.1	两层执行层级架构	4
5.2	参数协议与调用生命周期	5
5.3	<code>_SET_PLAN_</code> 计划选择	5
5.4	张量布局约定	5
5.5	多线程多卡 MultiGpuMultigrid	5
5.6	h5py 持久化体系	5
5.7	测试体系	6

6 关键代码/文档片段

6

7 参考源清单

6

8 结论与局限

7

1 仓库概览

PyQCU 是 Lattice QCD 的 Python/Cython 库: CUDA 加速的 Wilson/Clover Dirac 算子、BiStabCG 与多重网格 (MultiGrid) 求解器、stout smearing 与规范场生成, 经 MPI 分布于 4D 进程网格 (AGENTS.md:1-10)。git 元信息: 分支 main, HEAD = a456932, 标签体系 stab<N>/dev<N>/bug<N>, 初始提交 0204fa5 (实测 git rev-parse)。

1.1 规模统计与覆盖

下表为排除 .git/、.opencode/、外部镜像 refer/git-rep/、logs/ 历史产物与 examples/ 日志后的实测统计 (子代理 find 实测)。

指标	实测值
Python (.py)	121
CUDA 内核 (.cu)	30
C++ 头 (.h)	24
文档 (.md)	45
仓库总文件 (排除噪声)	325
标签数	132

表 1: 仓库实测统计 (数据来源: find/wc/git 实测, 2026-08-18)

饱和覆盖登记表 (四条遍历路径交叉: 目录/类型/git 历史/入口引用链):

目录	文件数	状态与职责
pyqcu/	26 .py + 13 .md	精读: 纯 Python 算子/求解器/smear/lattice/cuda 桥/工具/testing
cpp/cuda/qcu/src/	30 .cu	精读: CUDA 内核 + 启动封装 (22 个 C API 对应)
cpp/cuda/qcu/include/	23 .h	精读: 模板化内核头 (含 define.h)
cpp/cuda/qcu/python/	1 .h	精读: C API pyqcu.h (须与 pxd 同步)
cpp/{cann,dtk,maca}/	占位	仅索引: PASS stub, 无实现
examples/	95 .py + 11 .md	浏览: 33 个 conftest 入口 + 后端测试脚本
docs/	7 .md	浏览: 维度/环境/安装/示例/剖析说明

表 2: 饱和覆盖登记表 (数据来源: 全库索引实测)

2 主体结构

PyQCU 呈清晰的三层分层：纯 Python 参考实现层、Cython 桥接层、C++ CUDA 生产后端层，外加兼容层与文档/产物层。

层次	目录	职责
入口层	pyqcu/, examples/, docs/, logs/	库入口、测试入口、文档、产物归档
核心层	pyqcu/dslash/, pyqcu/solver/, pyqcu/smear/	纯 Python 算子/求解器/smear (PyTorch)
桥接层	pyqcu/cuda/	Cython 桥 qcu.pyx + 参数常量 define.py + 多线程多卡
后端层	cpp/cuda/qcu/	26 个模板头 + 30 个 .cu + C API pyqcu.h
兼容层	pyqcu/cann/, cpp/{cann, dtk, maca}/	昇腾 NPU 兼容 (实) / 其他后端占位 (PASS)
参考层	refer/	外部库镜像 (DDalphaAMG/PyQUDA/quda) , 参考背景

表 3: 主体结构分层 (数据来源: 全库索引实测 + AGENTS.md:14-17)

两层执行层级是设计主轴 (AGENTS.md:14-17) : 纯 Python 层 (pyqcu/dslash/, solver/, smear/) 为 PyTorch 实现, 可跑 CPU/CUDA/昇腾 NPU (经 pyqcu.cann 兼容层); C++ CUDA 后端 (cpp/cuda/qcu/) 为手写内核 + MPI halo 交换, 经 Cython 桥暴露, 是生产路径。

3 各部分关系

依赖主链为「纯 Python 层 → Cython 桥 → C++ CUDA 后端」, 由参数协议 params/argv/set_ptrs 串联 (AGENTS.md:21)。

要点

依赖主链: pyqcu/dslash/_wilson.py → pyqcu/cuda/qcu/qcu.pyx → cpp/cuda/qcu/src/wilson_dslash.cu → cpp/cuda/qcu/include/lattice_wilson_dslash.h

- **协议同步链:** pyqcu/cuda/define.py:11-90 与 cpp/cuda/qcu/include/define.h:43-122 一一对应 (_LAT_X=0, _SET_INDEX=15, _SET_PLAN=16 等) , 任一失配导致静默内存损坏 (AGENTS.md:21)。
- **桥接别名链:** qcu.pyx:1 from qcu_api cimport X as _c_X 避免与 pyx def 同名覆盖, qcu_api.pxd:1-23 全部带 nogil (子代理核实)。

- **混合求解链**：多重网格可混合两层——最细层平滑用 C++ 后端 (`with_cuda_qcu=True`)，更粗层用纯 Python (`AGENTS.md:28` 隐含)。
- **持久化链**：`pyqcu/tools/_io.py` 的 `save_tensor_h5/load_tensor_h5` 与 MPI `mpio` 路径，被 `_multi_gpu.py:139-143` 的粗算子缓存复用。

4 项目思路

4.1 设计意图

双路径设计（参考实现 + 生产后端）兼顾**正确性**（Python 层可跨平台验证物理）与**性能**（C++ CUDA 内核生产路径）。参数协议用三张扁平张量而非复杂对象，便于 Python 与 C++ 零拷贝桥接 (`AGENTS.md:21`)。

4.2 演进脉络

从初始提交 `0204fa5` 到 `stab27` 共 1120 提交；自有库代码主导文件为 `cpp/cuda/qcu/include/lattice_clover_multigrid.h` (+1833)、`cpp/cuda/qcu/src/clover_dslash_multi.cu` (+1610)、`cpp/cuda/qcu/src/wilson_dslash.cu` (+1442) 与 `pyqcu/tools/_multigrid.py` (+819)、`pyqcu/cuda/_multi_gpu.py` (+568)，显示项目从 Wilson dslash 演进到 Clover + 多重网格 + 多线程多卡 (`diff init→stab27` 实测)。

4.3 典型 workflow

一次典型任务：`source ./env.sh` → 构建 `libqcu.so` (`build.sh`) → 构建 Cython 扩展 (`install.sh`) → 在 `examples/pyqcu/conftest.py` 解锁测试 → 运行。生产调用生命周期：`applyInitQcu` → 操作 → `params[_SET_INDEX_] += 1` → `applyEndQcu` (`AGENTS.md:23`)。

项目思路

这个仓库之所以长成「纯 Python 参考 + C++ 生产 + Cython 薄桥 + 多线程多卡」的形态，是为了在保持物理实现可验证的同时，用 GPU 内核与一线程一卡并行逼近生产级格点 QCD 求解性能。

5 主题分析

5.1 两层执行层级架构

纯 Python 层约定所有代码 `import pyqcu.cann as _torch`，不得直接 `import torch`——NPU 不支持复数张量，`cann` 层自动分解实虚部 (`AGENTS.md:16`)。C++ 后端经 `pyqcu/cuda/qcu/qcu.pyx` 暴露 (`AGENTS.md:17`)。

5.2 参数协议与调用生命周期

三个扁平张量桥接 Python C++ (pyqcu/cuda/define.py:8,82,90) : params(int32[54])、argv(float[7])、set_ptrs(int64[100])。关键不变量：连续 C++ 调用间必须 params[define._SET_INDEX_]+=1 (AGENTS.md:23)，否则 scratch 缓冲复用冲突、结果错误。

```

1 _DATA_TYPE_SIZE_ = 10
2 _SET_INDEX_ = 15
3 _SET_PLAN_ = 16
4 _SET_PLAN_N_2_ = -2 # just for laplacian
5 _SET_PLAN_N_1_ = -1 # just for gauss gauge
6 _SET_PLAN0_ = 0 # for wilson dslash
7 _SET_PLAN1_ = 1 # just for bistabcg and cg and the whole dslash for them
8 _SET_PLAN2_ = 2 # for clover dslash

```

5.3 __SET_PLAN__ 计划选择

define.py:38-43 给出算子类型：-2 Laplacian、-1 Gauss gauge、0 Wilson dslash、1 BiStabCG/CG、2 Clover dslash（子代理核实 define.py:38-43）。

5.4 张量布局约定

时空维永远是最后 4 轴 (...xyzt) , ward 索引用负整数 (wards['x']==-4) : 规范场 [3,3,4,Lx,Ly,Lz,Lt]、费米子场 [4,3,Lx,Ly,Lz,Lt]、Clover 项 [4,3,4,3,Lx,Ly,Lz,Lt] (AGENTS.md:35-37; pyqcu/lattice/__init__.py:16-20 定义 wards)。

5.5 多线程多卡 MultiGpuMultigrid

pyqcu/cuda/_multi_gpu.py 的 MultiGpuMultigrid 单进程内 N 线程 × 卡绑定并行；每线程独立 params/argv/set_ptrs 副本 (_SET_INDEX_ 各自从 0 计数)，要求单 MPI rank（子代理核实 _multi_gpu.py:234-242）。

```

1 from mpi4py import MPI
2 # C++ LatticeSet::init 用 MPI_COMM_WORLD 真实 rank 覆盖 _NODE_RANK_,
3 # 多线程独立实例语义要求单 rank (多进程分布走每进程单线程实例路径)。
4 if MPI.COMM_WORLD.Get_size() > 1:
5     raise RuntimeError(
6         "PYQCU::CUDA::MULTI_GPU:\n MultiGpuMultigrid requires a single "
7         "MPI rank (mpirun -np 1): multi-process distribution uses "
8         "per-rank single-thread instances; multi-GPU parallelism inside "
9         "one process uses one-thread-one-GPU.")

```

5.6 h5py 持久化体系

pyqcu/tools/_io.py 的 save_tensor_h5/load_tensor_h5 每调用独立 File 句柄、多线程安全（子代理核实 _io.py:165-194）；粗算子缓存 _multi_gpu.py:139-143 单句柄一次写全部 dataset，勿逐 dataset 覆盖重建 (AGENTS.md:28)。

5.7 测试体系

测试函数集中在 `pyqcu/testing/__init__.py` (`test_dslash_wilson:38`、`test_dslash_clover:202`、`test_solver:292`、`test_multigrid_multithread:576`、`test_multi_gpu_multigrid:620` 等), 各 `examples/*/confctest.py` 为 `pytest` 入口 (`AGENTS.md:26`)。

6 关键代码/文档片段

Wilson dslash 的 hopping 项 (纯 Python 实现, 夸克相邻格点跃迁):

```

1      # Term 1: (1 - _) U_{x,} src_{x+}
2      src_plus = _torch.roll(src, shifts=-1, dims=ward)
3      U_src_plus = _torch.einsum('Ccxyzt,scxyzt->sCxyzt', U_mu, src_plus)
4      term1 = _torch.einsum(
5          'Ss,sCxyzt->SCxyzt', I_minus_gamma[ward_key], U_src_plus)
6      # Term 2: (1 + _) U_{x-,}^† src_{x-}
7      src_minus = _torch.roll(src, shifts=1, dims=ward)
8      U_dag_minus = _torch.roll(U_dag_mu, shifts=1, dims=ward)
9      U_dag_src_minus = _torch.einsum(
10         'Ccxyzt,scxyzt->sCxyzt', U_dag_minus, src_minus)
11     term2 = _torch.einsum(
12         'Ss,sCxyzt->SCxyzt', I_plus_gamma[ward_key], U_dag_src_minus)
13     # Combine terms and subtract from dest
14     hopping = term1 + term2
15     dest -= _factor * hopping

```

Cython 桥 with `nogil` 真并行入口:

```

1 def applyInitQcu(_set_ptrs, _params, _argv):
2     cdef long long set_ptrs, params, argv
3     set_ptrs = _set_ptrs.contiguous().data_ptr()
4     params = _params.contiguous().data_ptr()
5     argv = _argv.contiguous().data_ptr()
6     with nogil:
7         _c_applyInitQcu(set_ptrs, params, argv)

```

7 参考源清单

参考源	类型	内容/作用
AGENTS.md:14-17	仓库	两层执行层级定义
AGENTS.md:21	仓库	参数协议 <code>params/argv/set_ptrs</code> 与同步要求
AGENTS.md:23	仓库	调用生命周期与 <code>__SET_INDEX__</code> 递增不变量
AGENTS.md:35-37	仓库	张量布局约定
pyqcu/cuda/define.py:8-90	仓库	<code>params/argv/set_ptrs</code> 常量定义
pyqcu/cuda/define.py:36-43	仓库	<code>__SET_PLAN__</code> 取值含义
pyqcu/lattice/__init__.py:16-20	仓库	<code>wards</code> 负索引表

参考源	类型	内容/作用
pyqcu/cuda/_multi_gpu.py:234-242	仓库	单 MPI rank 强制
pyqcu/cuda/qcu/qcu.pyx:1,10-16	仓库	Cython 桥 nogil 调用
pyqcu/dslash/_wilson.py:49-63	仓库	Wilson hopping 项实现
cpp/cuda/qcu/include/define.h:43-122	仓库	C++ 端参数协议镜像
pyqcu/tools/_io.py:165-194	仓库	h5py 持久化 API
pyqcu/testing/__init__.py:16-620	仓库	集成测试套件

8 结论与局限

结论

结构：PyQCU 以「纯 Python 参考层 Cython 桥 C++ CUDA 生产层」三层组织，兼容层覆盖 NPU/占位后端。关系：两层由参数协议 `params/argv/set_ptrs` 与 `_SET_PLAN_` 计划选择织成依赖网，桥接别名与 `pxd/pyqcu.h` 同步是正确性的关键锁点。思路：双路径 + 一线程一卡并行，在可验证性与生产性能之间取平衡；多重网格（Galerkin + R3 fix + 5-stream）是核心特色。

参考背景

参考目录 `refer/`（1559 文件，外部库镜像）与 `docs/`（7 文件）在本分析中作为背景引入；`refer/` 提供 QUDA/PyQUDA 对照，但非 PyQCU 自有代码。

局限与风险

未覆盖/局限：**未验证** `cpp/cuda/qcu/src/` 全部 30 个内核的执行细节（仅抽样精读代表文件）；`examples/` 95 个测试脚本为浏览态，未逐文件解析；多线程多卡的实际性能数据未在本次静态分析中实测（见 diff 报告与 pure 报告）。

PyQCU 核心算法穷尽剖析

代码—物理双向映射：Wilson/Clover dslash、BiStabCG、多重网格与一线程一卡并行

opencode pure

2026-08-18

摘要

本报告对 PyQCU 核心算法做穷尽剖析，判定项目为**代码—物理交叉**性质（判定依据：C++ CUDA 内核 + 纯 Python 实现 + SU(3) 规范物理等量共存）。穷尽枚举 8 个核心候选，三态分配为深挖 7 / 浅析 1 / 排除若干。每个深挖部分给出代码—物理双向映射与唯一竞争力证据。最强竞争力：多重网格（Galerkin 宽 stencil + 粗校正后 BiCGStab 状态重置 R3 fix + 5-stream + 混合并行）与一线程一卡 **with nogil** 真并行。

目录

1 项目定位与核心部分清单	2
1.1 性质判定	2
1.2 穷尽候选清单（三态闭环）	2
2 核心部分逐深剖	2
2.1 C1 Wilson Dirac 算子（dslash）	2
2.2 C2 Clover Dirac 算子	3
2.3 C3 BiStabCG 求解器	4
2.4 C4 多重网格 V-cycle	5
2.5 C5 stout smearing（浅析）	6
2.6 C6 33-tensor stencil 构建 / 批量 BiCGStab	6
2.7 C7 CUDA 内核与 MPI halo 交换架构	6
2.8 C8 Cython 桥 nogil 真并行	7
3 独特性评估	7
4 参考源清单	7
5 结论与局限	8

1 项目定位与核心部分清单

1.1 性质判定

实测：`cpp/cuda/qcu/src/` 30 个 CUDA 内核（物理求解执行体）、`pyqcu/dslash/` 纯 Python 算子（含 Wilson 算子公式与 `wards` 负索引物理约定）、`pyqcu/lattice/` 的 `gamma/Gell-Mann/SU(3)` 数学对象，三者共存 → [交叉向](#)（代码 物理符号一一对应）。

1.2 穷尽候选清单（三态闭环）

编号	候选	三态（深挖/浅析/排除）
C1	Wilson Dirac 算子	深挖
C2	Clover Dirac 算子	深挖
C3	BiStabCG 求解器	深挖
C4	多重网格 V-cycle	深挖
C5	stout smearing	浅析
C6	33-tensor stencil / 批量 BiCGStab	深挖
C7	CUDA 内核与 MPI halo 交换架构	深挖
C8	Cython 桥 nogil 真并行	深挖
E1	<code>_gmres.py</code>	排除（占位 stub，未实现）
E2	<code>cpp/{cann,dtk,maca}/</code>	排除（PASS 占位，无实现）
E3	TileLang JIT (<code>_einsum</code> 等)	排除（可选降级，非核心）

表 1: 核心部分候选清单（穷尽枚举，三态填满；数据来源：全库索引 + 子代理核实行号）

计数：深挖 7（C1-C4,C6-C8）、浅析 1（C5）、排除 3（E1-E3），无未处理候选。

2 核心部分逐深剖

2.1 C1 Wilson Dirac 算子 (dslash)

是什么 Wilson-Dirac 算子把夸克场在相邻格点的跃迁编码为 $SU(3)$ link 与 Dirac γ 矩阵的收缩；奇偶预处理把 12×12 矩阵分裂为偶 奇耦合 (`_wilson.py:16`)。

代码—物理映射

代码符号	物理对象	参考源
<code>U_mu = U[...ward,...]</code>	方向 μ 的 $SU(3)$ link $U_{x,\mu}$	<code>_wilson.py:47</code>

代码符号	物理对象	参考源
I_minus_gamma/I_plus_gamma	$(1 \mp \gamma_\mu)$ 自旋投影	_wilson.py:38-39
_torch.roll(src, ∓ 1)	费米子沿 μ 的格点跃迁 $x \pm \mu$	_wilson.py:50,55
kappa/u_0	Wilson 质量 κ 与 tadpole 改进 u_0	_wilson.py:31
give_wilson_eo/oe	偶 $\rightarrow$ 奇/奇 $\rightarrow$ 偶子块 D_{oe}/D_{eo}	_wilson.py:74,152

唯一竞争力 纯 Python (CPU/CUDA/NPU) 与手写 CUDA 内核双路径并存; hopping 项显式拆为 $M^\pm$ 并 reshape 为 [12,12,X,Y,Z,T], 与 C++ 按方向打包的 hopping 张量同构 (子代理核实 _wilson.py:230-294)。

2.2 C2 Clover Dirac 算子

是什么 改进 Wilson 费米作用量的对角项 $M = I + c_{sw}\kappa \sum_{\mu < \nu} \sigma_{\mu\nu} F_{\mu\nu}$, $F_{\mu\nu}$ 由 4-plaquette (clover leaf) 构造, 消除 $O(a)$ 离散化误差 (_clover.py:13-15)。

代码—物理映射

代码符号	物理对象	参考源
U_mu, U_nu, U_dag_mu, U_dag_nu	plaquette 四角 link	_clover.py:141-144
F += einsum(U·U·U [†] ·U [†])	单 plaquette $\rightarrow$ 色电场强	_clover.py:236-240
F -= F [†] .T.conj()	反厄米化 $F - F^\dagger$	_clover.py:262
sigma = gamma_gamma[...]	$\sigma_{\mu\nu} = -i/2[\gamma_\mu, \gamma_\nu]$	_clover.py:108
_clover_factor=-0.125· κ /u_0	改进系数 c_{sw}	_clover.py:118
clover [4,3,4,3,XYZT]	块对角 Clover 项	_clover.py:101-102

关键片段 $F_{\mu\nu}$ 四 plaquette + 反厄米 + σ 收缩:

```

1     else:
2         # $$$U_1 &= u(x,\mu)u(x+\mu,\nu)u^{\dag}(x+\nu,\mu)u^{\dag}(x,\nu) \quad \backslash\backslash$$$
3         temp1 = _torch.einsum('abxyz,bcxyz->acxyz', U_mu,
4                                _torch.roll(U_nu, shifts=-1, dims=mu))
5         temp2 = _torch.einsum('abxyz,bcxyz->acxyz', temp1,
6                                _torch.roll(U_dag_mu, shifts=-1, dims=nu))
7         F += _torch.einsum('abxyz,bcxyz->acxyz', temp2, U_dag_nu)
8         # $$$U_2 &= u(x,\nu)u^{\dag}(x-\mu+\nu,\mu)u^{\dag}(x-\mu,\nu)u(x-\mu,\mu) \quad \backslash\backslash$$$
9         temp1 = _torch.einsum('abxyz,bcxyz->acxyz', U_nu,
10                                _torch.roll(_torch.roll(U_dag_mu, shifts=1, dims=mu), shifts=-1, dims=nu))
11         temp2 = _torch.einsum('abxyz,bcxyz->acxyz', temp1,
12                                _torch.roll(U_dag_nu, shifts=1, dims=mu))
13         F += _torch.einsum('abxyz,bcxyz->acxyz', temp2,
14                                _torch.roll(U_mu, shifts=1, dims=mu))
15         # $$$U_3 &= u^{\dag}(x-\mu,\mu)u^{\dag}(x-\mu-\nu,\nu)u(x-\mu-\nu,\mu)u(x-\nu,\nu) \quad \backslash\backslash$$$
16         temp1 = _torch.einsum('abxyz,bcxyz->acxyz', _torch.roll(U_dag_mu, shifts=1, dims=mu),
17                                _torch.roll(_torch.roll(U_dag_nu, shifts=1, dims=mu), shifts=1, dims=nu))
18         temp2 = _torch.einsum('abxyz,bcxyz->acxyz', temp1,

```

```

19         _torch.roll(_torch.roll(U_mu, shifts=1, dims=mu), shifts=1, dims=nu))
20     F += _torch.einsum('abxyzt,bcxyzt->acxyzt', temp2,
21         _torch.roll(U_nu, shifts=1, dims=nu))
22     # $$U_4 \&= u^{\{\dag\}}(x-\nu, \nu)u(x-\nu, \mu)u(x-\nu+\mu, \nu)u^{\{\dag\}}(x, \mu) \\$
23     temp1 = _torch.einsum('abxyzt,bcxyzt->acxyzt', _torch.roll(U_dag_nu, shifts=1, dims=nu),
24         _torch.roll(U_mu, shifts=1, dims=nu))
25     temp2 = _torch.einsum('abxyzt,bcxyzt->acxyzt', temp1,
26         _torch.roll(_torch.roll(U_nu, shifts=-1, dims=mu), shifts=1, dims=nu))
27     F += _torch.einsum('abxyzt,bcxyzt->acxyzt', temp2, U_dag_mu)
28     # Give whole F
29     F -= F.permute(1, 0, 2, 3, 4, 5).conj() # -BEFORE^{\dag}
30     # Multiply F with sigma
31     sigmaF = _torch.einsum(
32         'Ss,Ccxyzt->SCscxyzt', sigma, F)
33     # Make Clover term
34     clover += _clover_factor * sigmaF # -0.125 * kappa/u_0 * sigmaF

```

唯一竞争力 批量 12×12 求逆（禁止逐点 for 循环，_clover.py:314-327）；Clover 项形状严格为 $[4, 3, 4, 3, X, Y, Z, T]$ （AGENTS.md:37）。

2.3 C3 BiStabCG 求解器

是什么 求解 $D\psi = \eta$ 的 BiCGStab 迭代，残差双正交于影子残差 $\tilde{r}$ ，每步两 matvec (_bistabcg.py: 8-91)。

代码—物理映射 matvec 狄拉克算子 D ； $r = b - \text{matvec}(x)$ 残余 $r = \eta - D\psi$ ； $\rho = \langle \tilde{r}, r \rangle$ ； $\alpha = \rho / \text{rtv}$ 步长 α ； $\omega = \langle t, s \rangle / \langle t, t \rangle$ 稳定化 ω （子代理核实 _bistabcg.py:36-66）。

关键片段

```

1     for i in range(max_iter):
2         iter_start_time = perf_counter()
3         # BUGFIX 2026-07-28 R2: BiCGStab breakdown detection.
4         # rho == 0 means r_tilde == r (method has lost orthogonality).
5         # vdot(r_tilde, v) == 0 is a pivot breakdown (division by zero).
6         # vdot(t, t) == 0 means t == 0 (rare lucky breakdown, or stagnation).
7         rho = tools.vdot(r_tilde, r)
8         if abs(rho) < 1e-30:
9             raise RuntimeError(
10                 f"BiCGStab breakdown at iter {i}: rho == 0 "
11                 f"(r_tilde orthogonal to r). The method cannot continue.")
12         beta = (rho / rho_prev) * (alpha / omega)
13         rho_prev = rho
14         p = r + beta * (p - omega * v)
15         v = matvec(p)
16         rtv = tools.vdot(r_tilde, v)
17         if abs(rtv) < 1e-30:
18             raise RuntimeError(
19                 f"BiCGStab breakdown at iter {i}: vdot(r_tilde, v) == 0 "
20                 f"(pivot breakdown). The method cannot continue.")
21         alpha = rho / rtv

```

```

22     s = r - alpha * v
23     t = matvec(s)
24     tts = tools.vdot(t, t)
25     if abs(tts) < 1e-30:
26         raise RuntimeError(
27             f"BiCGStab breakdown at iter {i}: vdot(t, t) 0 "
28             f"(t is zero or near-zero). The method cannot continue.")
29     omega = tools.vdot(t, s) / tts
30     x = x + alpha * p + omega * s
31     r = s - omega * t

```

唯一竞争力 2026-07-28 起 breakdown 改自动重启（保留 x/r ，重置影子向量）；C++ 后端标量 $\rho/\alpha/\omega/\beta$ 全留 device，迭代内无 host→device 标量拷贝（_SET_PLAN_=1, define.py:42）。

2.4 C4 多重网格 V-cycle

是什么 自适应多重网格：细层奇偶预处理 Clover 平滑（BiCGStab）+ 粗网格校正；近零空间向量经 Galerkin 投影 $A_c = P^T D_{fine} P$ 构造粗算子，递归消除低频误差（_multigrid.py:204-521）。

代码—物理映射 lonv_list 近零空间（低频模态）基 P ；restrict/prolong P^T/P ；cycle(level) V-cycle；e_fine=prolong(e_coarse) 粗解提升校正；v_cycle(lev+1) 递归粗层（lattice_clover_multigrid.h:1350）。

关键片段 粗校正后 BiCGStab 状态重置（R3 fix，避免 $\rho = \langle \tilde{r}_{old}, r_{new} \rangle$ 无意义）：

```

1         x = x + e_fine
2         r = b - matvec(x)
3         count_restart = 0
4         # BUGFIX 2026-07-28 R3: BiCGStab state must be fully reset after
5         # a coarse-grid correction. The residual r has changed, so the
6         # shadow vector r_tilde, the search directions p/v/s/t, and the
7         # scalar coefficients rho_prev/alpha/omega must all be reinitialized.
8         # Without this, rho = vdot(r_tilde_old, r_new) gives meaningless
9         # results, causing convergence degradation or divergence.
10        r_tilde = r.clone()
11        p = torch.zeros_like(b)
12        v = torch.zeros_like(b)
13        s = torch.zeros_like(b)
14        t = torch.zeros_like(b)
15        rho = torch.tensor(1.0, dtype=b.dtype, device=b.device)
16        rho_prev = torch.tensor(1.0, dtype=b.dtype, device=b.device)
17        alpha = torch.tensor(1.0, dtype=b.dtype, device=b.device)
18        omega = torch.tensor(1.0, dtype=b.dtype, device=b.device)

```

唯一竞争力 Galerkin 既支持纯 Python 探测也支持 33-tensor 宽 stencil（耦合同格点 + 8 近邻 + 24 对角）；最粗层用 cooperative 融合单 kernel BiStabCG；5-stream 架构；with_cuda_qcu 混合并行（细层 C++、更粗纯 Python）。

2.5 C5 stout smearing (浅析)

是什么 Stout 规范场涂抹：每步 $U_{new} = e^Q \cdot U$, Q 为反厄米 $\text{su}(3)$ 元 (Morningstar-Peardon 投影, `_stout.py:13-204`)。

代码—物理映射 $Q_{\mu} \text{su}(3)$ 涂抹生成元; `0.5j*(Q.conj().T-Q)` 反厄米化; `c0=Tr(Q3)/3`, `c1=Tr(Q2)/2` 投影不变量; halo 每步重算 修复 `nstep>1` 不更新 U 的反模式 (子代理核实 `_stout.py:100-130`)。

定位 重要但属预处理辅助, 非核心求解算子, 故浅析。

2.6 C6 33-tensor stencil 构建 / 批量 BiCGStab

是什么 逐格点探测 $A_c = P^T S P$ 的「宽」stencil (8 近邻 + 24 对角, 因 Schur 算子耦合格点 $\pm 2\mu$ 与 $\mu \pm \nu$ 对角); 批量 BiCGStab 同时对 batch 维全部右端求解 (`_multigrid.py:277-819`)。

代码—物理映射 `sit[E,E,Xc,Yc,Zc,Tc]` on-site 块 $A_c(x, x)$; `hop_nn[2,4,E,E,...]` 近邻耦合 $A_c(x, x \pm \mu)$; `hop_diag[2,2,6,E,E,...]` 对角耦合 $A_c(x, x \pm \mu \pm \nu)$; `_bistabcbg_batch` 多右端近零空间迭代。

唯一竞争力 批量版把 null 向量生成从逐场 C++ (40min+) 提速到分钟级 (8x8x8x16 约 135s→15s, 10x; 子代理核实 `_multigrid.py:751-755`)。

2.7 C7 CUDA 内核与 MPI halo 交换架构

是什么 手写 CUDA 内核 + 4D 进程网格 MPI halo 交换, 构成 `dslash` 与 MG 的执行骨架 (`lattice_wilson_dslash.h:17-602`)。

唯一竞争力 5-stream 架构 (MG, `lattice_clover_multigrid.h:27-30`) ; `dslash` 单 rank 快速路径 (`grid==[1,1,1,1]` 时省去中间同步, :321-601) ; MPI halo 用阻塞 `MPI_Sendrecv` (`dslash`) / `MPI_Isend/Irecv`; 标量 `MPI_Allreduce`。 `define.h:43-122` 与 `define.py:11-90` 严格同步。

关键片段 `dslash` 单 rank 快速路径判定与末尾单次 sync:

```

1  if (!is_coarsest && count_restart >= st.num_restart) {
2      // ---- Coarse-grid correction to the next level ----
3      restrict_op(levels[lev+1].rhs, st.r, lev);
4      zero_c(levels[lev+1].x, lev+1);
5      checkCudaErrors(cudaStreamSynchronize(S));
6
7      v_cycle(lev+1);

```

2.8 C8 Cython 桥 nogil 真并行

是什么 `qcu.pyx` 把 `pyqcu.h` 的 C 函数包成 Python 可调用；GIL 段取张量裸指针 → `with nogil` 调 C++，多线程可真并行进入 `libqcu.so` (`qcu.pyx:1-215`)。

代码—物理映射 `with nogil: _c_applyInitQcu(...)` 释放 GIL 进入后端(真并行前提)；`cdef long long ...data_ptr()` 张量裸指针桥接 (`params/argv/set_ptrs` 协议)；`qcu_api.pxd` 别名 `cimport` 避免与 `pyx def` 同名覆盖导致 `nogil` 失效；`MultiGpuMultigrid` 一线程一卡并行 (单 MPI rank)。

关键片段

```
1 def applyInitQcu(_set_ptrs, _params, _argv):
2     cdef long long set_ptrs, params, argv
3     set_ptrs = _set_ptrs.contiguous().data_ptr()
4     params = _params.contiguous().data_ptr()
5     argv = _argv.contiguous().data_ptr()
6     with nogil:
7         _c_applyInitQcu(set_ptrs, params, argv)
```

唯一竞争力 每线程独立 `params/argv/set_ptrs` 副本 + `torch.cuda.set_device` 绑定，GIL 释放后真并行 (`_multi_gpu.py:8-14,40-42`)。

3 独特性评估

核心部分	独特性	证据
C4 多重网格	强	R3 fix + 5-stream + 混合并行 + 33-tensor 宽 stencil
C8 一线程一卡	强	nogil + 每线程独立副本 + 单 MPI rank
C3 BiStabCG	中	自动重启 breakdown + device 标量
C1/C2 dslash	中	双后端同构 + 批量逆 (禁 for 循环)
C6 批量构建	强	10× 提速 null 向量生成
C7 CUDA/MPI	中	单 rank 快速路径 + 5-stream

4 参考源清单

参考源	类型	内容/作用
<code>pyqcu/dslash/_wilson.py:16,47-63</code>	仓库	Wilson 算子公式与 hopping
<code>pyqcu/dslash/_clover.py:13-15,101-327</code>	仓库	Clover 项构造与批量逆
<code>pyqcu/solver/_bistabcg.py:8-91</code>	仓库	BiCGStab 迭代体
<code>pyqcu/solver/_multigrid.py:204-521</code>	仓库	V-cycle 与 R3 fix

参考源	类型	内容/作用
pyqcu/smeat/_stout.py:13-203	仓库	stout 涂抹与投影
pyqcu/tools/_multigrid.py:277-819	仓库	33-tensor 与 批 量 BiCGStab
cpp/cuda/qcu/include/lattice_wilson_dslash.h:17-602	仓库	dslash 内核 + halo
cpp/cuda/qcu/include/lattice_clover_multigrid.h:27-1350	仓库	MG 5-stream + V-cycle
pyqcu/cuda/qcu/qcu.pyx:1-215	仓库	Cython 桥 nogil 调用
pyqcu/cuda/_multi_gpu.py:1-537	仓库	一线程一卡 MultiGpu-Multigrid

5 结论与局限

结论

PyQCU 最强竞争力在「多重网格 (Galerkin 宽 stencil + R3 fix + 5-stream + 混合并行)」与「一线程一卡 with nogil 真并行」两项。Wilson/Clover dslash 以双后端同构 + 批量逆保证正确与性能; BiStabCG 以设备端标量 + 自动重启取胜。穷尽 8 候选, 三态闭环 (深挖 7 / 浅析 1 / 排除 3)。

局限与风险

未覆盖/局限: CUDA 内核逐行执行语义未全量验证 (抽样代表文件); 33-tensor 与批量 BiCGStab 的实际加速比未在本次静态报告中实测; E1-E3 排除项未展开 (占位/可选)。所有行号经子代理 read/grep 核实, 但建议对 lattice_clover_multigrid.h 等大体量头文件做二次复核。

PyQCU 核心算法深剖（补充）：多重网格 V-cycle

15 步全流程重现（代码—物理交叉框架 A）

opencode pure

2026-08-18

摘要

本报告作为 `pure_pyqcu_20260818` 的补充，对 PyQCU 最强竞争力——多重网格 (MultiGrid) V-cycle 求解器——按「分析思路框架」A（代码/软件工作）15 步做全流程深剖。证据均来自当前 HEAD (`a456932`) 逐行 `read/gzip` 核实。核心结论：V-cycle 以近零空间向量 (null space) 经 Galerkin 投影 $A_c = P^T D_{fine} P$ 构造粗算子，递归消除低频误差；细层用 C++ CUDA 后端 BiCGStab 平滑 (5-stream 架构)，粗层纯 Python；粗网格校正后强制重置 BiCGStab 全部状态 (R3 fix) 并支持 breakdown 自动重启 (R2 演进)。实测收敛数据引自 `pyqcu/solver/AGENTS.md`。

目录

1 候选归属与性质判定

本报告聚焦 `pure_pyqcu_20260818` 清单中的 C4（多重网格 V-cycle，深挖）。项目性质为**代码—物理交叉**：V-cycle 的每个代码符号 (`lonv_list`、`restrict/prolong`、`cycle(level)`、`v_cycle`) 均有唯一物理对象（低频模态基 P 、限制/延拓、 D_l 层算子、递归粗层），故以 A 框架为主体、B 元素（物理图像）并入各步。

2 工作全流程重现（A 代码工作框架）

2.1 A1 目的与前期准备

为什么存在：求解格点 QCD 狄拉克方程 $D\psi = \eta$ 时，细层 Dirac 算子 D 的条件数随体积 $\propto V$ ，普通 Krylov 法（如 BiCGStab）在红外（低频）模态上收敛极慢；多重网格用粗网格显式消除低频误差，将迭代次数与体积解耦（`pyqcu/solver/AGENTS.md`「multigrid 类」）。**前置条件**：规范场 U 、Clover 项、 κ/u_0 、近零空间向量；生产路径需先 `build.sh+install.sh` 构建 `libqcu.so` 与 Cython 扩展（`AGENTS.md:10`）。

2.2 A2 输入

输入	类型	参考源
U （规范场）	[3,3,4,X,Y,Z,T]	_multigrid.py:241
clover_ee/oo	块对角 Clover	_multigrid.py:243-244
κ, u_0	标量	AGENTS.md 构造参数
lonv（近零空间）	迭代代生成	_multigrid.py:273-278
b （右端）	费米子场	_multigrid.py:340,524

2.3 A3 输出（应是什么）

解 x （费米子场， ψ ），以及 `convergence_history`（每轮残差，用于 `adaptive` 停滞判定与绘图，_multigrid.py:368,434,484）。正确性判据：细层残差降至 `tol`（默认绝对容差，_multigrid.py:369）。

2.4 A4 整体框架

多层递归：层 l 上先 BiCGStab 平滑，再做粗网格校正——将残差限制到层 $l+1$ 、递归 `cycle(l+1)` 求粗解、延拓回细层校正；最细层（ $l = 0$ ）走 C++ CUDA 后端（`with_cuda_qcu=True`），层 1 走 C++ 粗网格 `dslash`，层 2+ 纯 Python `einsum`（`pyqcu/solver/AGENTS.md`「执行层」）。递归入口 _multigrid.py:455 `self.cycle(level=level+1)`。

2.5 A5 关键细节

- **Galerkin 粗算子**：`operator()` 用 `prolong+matvec+restrict` 单格点探测 $P^T D_{fine} P$ ，写入 `sitting.M` 与 `hopping.M_plus/minus_list`（_operator.py:157-226）。
- **R3 fix**：粗校正后 `x=x+e_fine` 使残差改变，必须重置 `r_tilde,p,v,s,t,rho,rho_prev,alpha,omega`（_multigrid.py:465-482）。
- **自动重启**：`breakdown(rho/rtv/tts<1e-20 或非有限)` 时保留 `x/r` 重置状态后继续（_multigrid.py:392-430）。
- **5-stream**（C++ 端）：`main/_a/_b/_c/_d` 并行 BiCGStab 子步骤（`lattice_clover_multigrid.h:20-31`）。

2.6 A6 任务如何正确执行

典型调用链（生产路径）：`multigrid.init()`（构建 `null` 空间与粗算子） $\rightarrow$ `multigrid.solve(b)` $\rightarrow$ `cycle(0)` 递归 $\rightarrow$ `multigrid.end()` 释放 `scratch`（_multigrid.py:204,523,290）。C++ 后端每次调用间必须 `params[_SET_INDEX_]+=1`（_multigrid.py:262），否则 `scratch` 冲突。

2.7 A7 实际执行情况

未验证（本静态分析会话未构建 `libqcu.so` / 无 GPU 运行）。以下为仓库既有实测记录（`pyqcu/solver/AGENTS.md`「CUDA 混合路径同步与 `breakdown` 重启」「多线程实例并行」）：

要点

文档化实测：8x8x8x8 1L 残差 8.7e-7; 8x8x8x16 2L 残差 7.8e-7 (自动重启后收敛不受影响); 2 线程 8x8x8x8 1L 并行各 6.3e-7/9.5e-7 均收敛; 3 实例循环显式 `end()+del+gc.collect()` 后全 9e-7 收敛。

2.8 A8 实际输出情况

上述实测下, 输出 x 满足 $\|Dx - b\| < \text{tol}$, 且 `convergence_history` 单调下降 (粗校正前后双采样, `_multigrid.py:434,484`)。未验证: 本次会话未亲自产生输出数值。

2.9 A9 结果分析

结合 A7 文档数据: 多层 (2L) 在 8x8x8x16 上残差 7.8e-7, 说明粗网格校正有效消除了 BiCGStab 单独难以收敛的低频模态; 自动重启 (R2) 使偶发 breakdown 不再中断求解, 收敛不受影响; R3 fix 避免了粗校正后 `rho=vdot(r_tilde_old,r_new)` 无意义导致的收敛退化/发散 (`_multigrid.py:468-473` 注释)。多线程实例并行要求显式资源管理, 否则隐式 GC 与 C++ 操作并发导致残差 $O(1)$ (`_multigrid.py:290-303`)。

2.10 A10 综合分析

V-cycle 是 PyQCU 与朴素 BiCGStab 的根本分野: 它将「红外慢模态」显式下推到粗网格用更少自由度求解。Galerkin 投影保证粗算子与细算子谱一致 (`_operator.py:158` 注释); 5-stream 与 device 端标量 (`cpp/.../AGENTS.md` 关键不变量 1) 保证 C++ 平滑的高吞吐; R3/R2 两处 fix 是工程鲁棒性的关键, 缺失其一即退化或发散。这与纯 Python 参考实现 (`_multigrid.py`) 形成「生产后端 + 可验证参考」双轨。

2.11 A11 结尾总结

结论

多重网格 V-cycle = 近零空间 Galerkin 粗化 + 递归粗校正 + BiCGStab 平滑; 其工程正确性依赖三处关键: Galerkin 谱一致性、粗校正后 R3 状态重置、breakdown 自动重启 (R2)。PyQCU 以「C++ CUDA 5-stream 细层 + 纯 Python 粗层」混合, 兼顾性能与可验证性。

2.12 A12 结尾评价

优点: 低频误差消除显著、收敛与体积解耦、双后端可交叉验证、R2/R3 提升鲁棒性。**可改进:** null 向量迭代每向量一个 LatticeSet, 大格子 ($16^3 \times 32$) 单实例 init 仍可能 OOM (`pyqcu/solver/AGENTS.md` 「多实例资源管理」); 粗算子 33-tensor 宽 stencil 构建成本需批量优化 (见 C6)。

PyQCU 改动概览 (init → stab27)

初始提交 0204fa5 → 最新标签 stab27 全量 git diff 结构化分析

opcode diff

2026-08-18

摘要

本报告对 PyQCU 从初始提交 0204fa5 到最新标签 stab27 的全部改动做只读结构化概览。全量 2138 文件、+1,163,982/-2 行、区间内 1120 提交；其中外部库镜像 refer/ (1559 文件) 与测试/产物 examples/、logs/ 占增行绝大部分，真正自有库代码集中在 pyqcu/ (+6847) 与 cpp/ (+16234)。边界检查：冲突标记 0、二进制 170、未检测重命名。

目录

1 改动概览

要点

范围: 初始提交 0204fa5 (init) → 标签 stab27 (全历史)

提交: 区间内 1120 个提交

统计: 2138 个文件, +1,163,982 / -2 行

分组: 新增 (全量) / 外部镜像 refer/ 1559 文件 / 测试与产物 examples+logs 384 文件

主导: refer/git-rep/ (外部库镜像) 与 examples/pyqcu/*.txt (运行日志) 占增行绝大部分

自有库: pyqcu/ 47 文件 +6847 行; cpp/ 78 文件 +16234 行

关注: 冲突标记 0; 二进制 170; 未检测重命名

2 规模与分布 (实测)

顶层目录	文件数	增行	说明
refer	1559	385,023	外部库镜像 (DDalphaAMG/PyQUDA/quda 等), 非自有代码
examples	136	723,046	测试入口与运行日志 (*.txt 占主导)
logs	261	29,394	测试/调试产物归档 (.gitignore 豁免但历史已提交)

顶层目录	文件数	增行	说明
cpp	78	16,234	C++ CUDA 后端（手写内核 + MPI halo）
pyqcu	47	6,847	纯 Python 实现 + Cython 桥 + cann 兼容层
.opencode	36	2,493	opencode 运行环境，与库无关
docs	12	730	文档与既有分析报告
根文件	8	215	setup.py/env.sh/build.sh/AGENTS.md 等
合计	2138	1,163,982	—

表 1: 全量 diff 规模按顶层目录（数据来源: git diff --numstat 0204fa5 stab27 实测）

3 PyQCU 自有库代码演进

真正自有库代码集中在 pyqcu/ 与 cpp/。pyqcu/ 子目录分布：

子目录	文件数	增行
tools	9	1846
cuda	11	1604
dslash	5	1099
solver	5	749
testing	2	697
smear	3	248
cann	4	286
lattice	2	250
其他	7	68

表 2: pyqcu/ 子目录分布（数据来源: git diff --numstat 实测）

4 主导文件（自有库，按增行）

文件	角色	增行
cpp/cuda/qcu/include/lattice_clover_multigrid.h	Clover 多重网格核心头	1833
cpp/cuda/qcu/src/clover_dslash_multi.cu	多重网格 Clover dslash 内核	1610
cpp/cuda/qcu/src/wilson_dslash.cu	Wilson dslash 内核	1442
cpp/cuda/qcu/src/multigrid.cu	多重网格 V-cycle 内核	981
cpp/cuda/qcu/src/clover_dslash_single.cu	单层 Clover dslash 内核	918
pyqcu/tools/_multigrid.py	纯 Python 多重网格工具	819